Designing and simulating a computer

Guided project (GP) – PART II

Objective

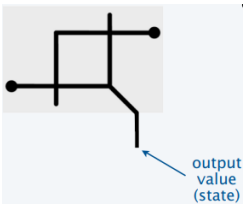
To simulate a quasi-design of miniaturized version of TOY i.e., TOY-8 using Logisim.

Context:

In this second part of the GP we would build the remaining modules: memory, registers, PC, IR, (essentially all of these are circuits that remember things) and then two other critical modules control and clock, and connect them together to realize our CPU!

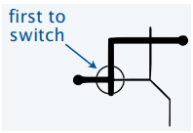
Steps:

16. Implement memory by realizing flip-flops:
    a. Interface of the flip-flop



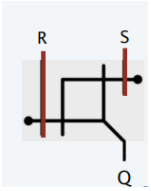
Where two switches each blocked by the other. Note sequence of switch operation matters.

    b. 0 is stored if first switch is switched first (see below), and it stays that way! (value remain intact)



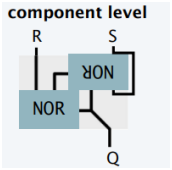
    c. 1 is stored if second switch is switched first (see below), and it stays that way i.e., value remain intact.

    d. Now adding two control pins to set and reset will establish control feedback

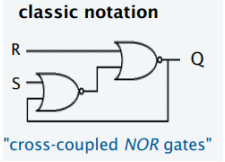


where S = set, R = reset and Q = bit value

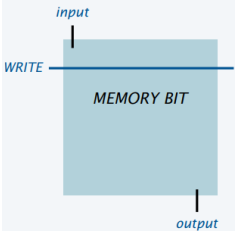
    e. Alternatively, at component level these can be realized with two NOR gates as shown below



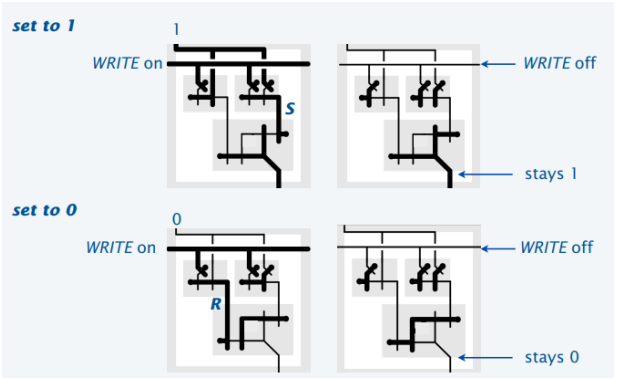
or in logical notation given as:



f. Now a memory bit interface shown below can be realized by adding an input instead of S, R controls by adding a WRITE control wire



g. Following diagram shows complete circuitry under the hood.

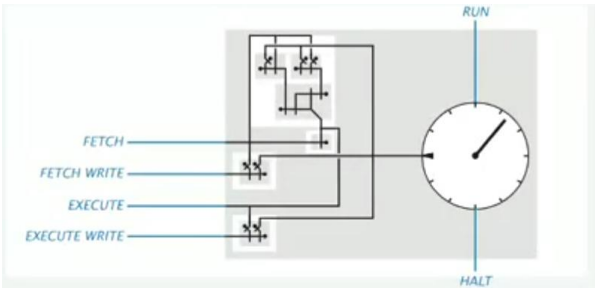


Note the use of 2 generalized AND gates.

NOTE

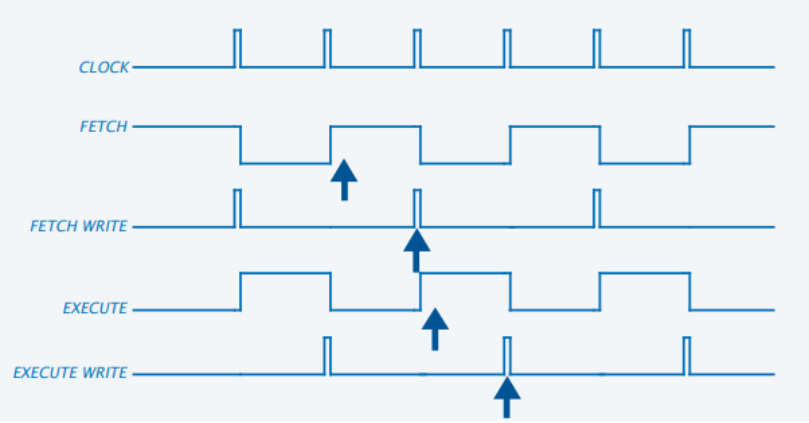
Now this memory bit module will be used not only in building memory bank but also in making register, PC and even fetch/execute block!

17. Implement Control wires and mating with clock
    a. Clock implements fetch/execute cycles
    b. Signals turn on control wires that change the state of PC, R, IR and memory
    c. Module circuit is shown below

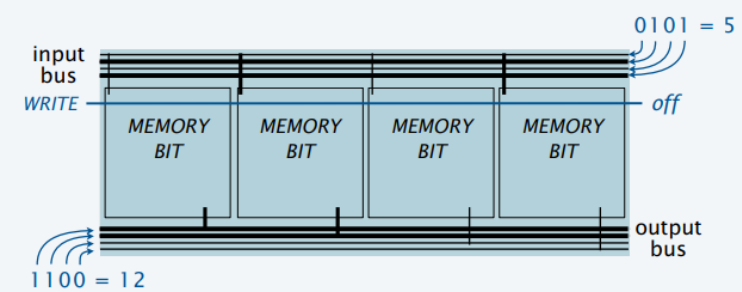


    d. Add AND gates to fetch/execute clock.
        i. FETCH WRITE = FETCH AND CLOCK
        ii. EXECUTE WRITE = EXECUTE AND CLOCK
    e. Use pre-built clock as its implementation is out of scope

f. Study the timing diagram below to make sense for the its function as the arrow shows the 4 distinct times when events take place in the cycle.



18. Implement the register
    a. Configuring bits



Please note these are the same bits that we made earlier.

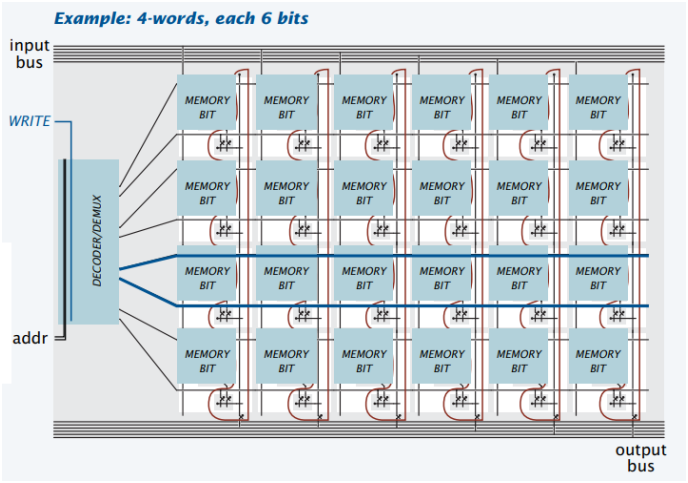
    b. Connect memory input bits to busses
    c. Use WRITE pulse for all of them
    d. All three registers:
        i. PC (holds 4-bit memory address),
        ii. IR (holds 8-bit instructions) and
        iii. R (which is general register to hold 8-bit data value)

    are built exactly like this. Please note the difference of bits and use correct number memory modules.

19. Implement 8-bit 16 words memory bank
    a. Example interface of 6-bit 4 words memory bank

Where 2 addr bits provide address to all 4 words each of 6 bit length. Note the input output busses and WRITE wire much like registers implemented earlier

b. The module circuit diagram is shown below



i. Decoder/Demux selects words

ii. One-hot muxes take selected word to output buses (both of these are already build)

c. Scale the above model to build 8-bit 16 words memory bank by

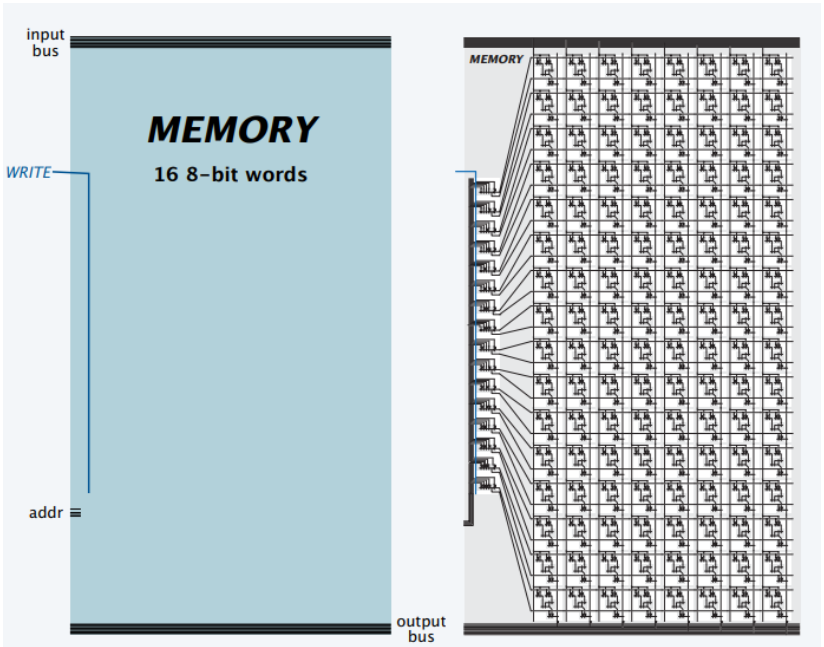
i. Adding 2 more memory module per word which were 6 in example above

ii. Adding 12 more words which are were merely 4 in the example above

iii. Using 4-to-16 decoder/demux, in the last example it was 2-to-4 decoder/demux

Note that now we have 4 addr pins to address all 16 words in the memory bank

iv. The completed memory bank will be like

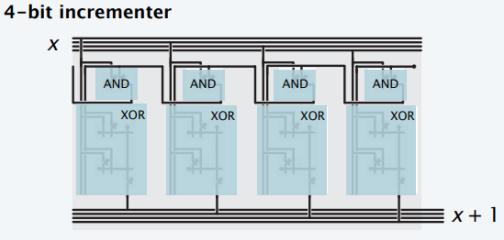


20. Implement PC

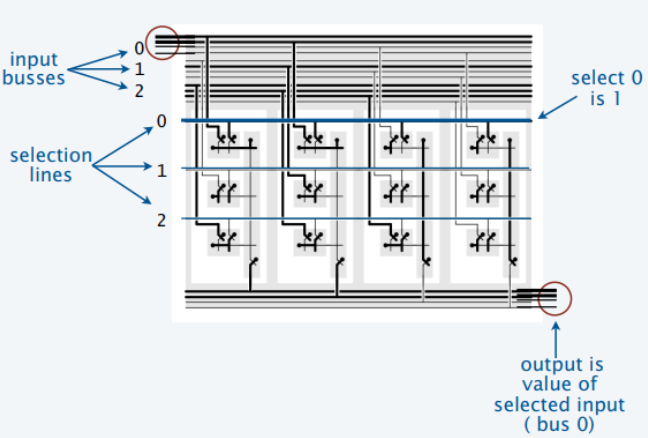
a. Components:

i. 4-bit register to hold address for next instruction (shown earlier)

ii. Incrementer to advance to next address adding 1 with each tick of the clock

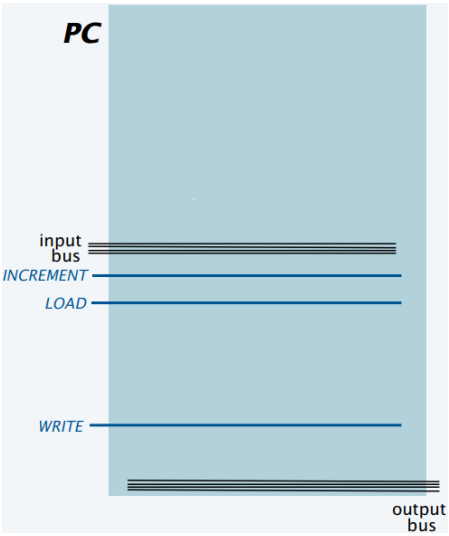


iii. 2-way bus mux as this will be required to take select input either from the incrementer or the memory in case of branch instruction. A 3-way mux circuit is shown below downsize it to a 2-way bus mux.



Note that sharing your output to multiple busses is hassle free as electrical signals travel in all directions with same speed but acquiring input from multiple buses require a circuit ***m-way bus mux***

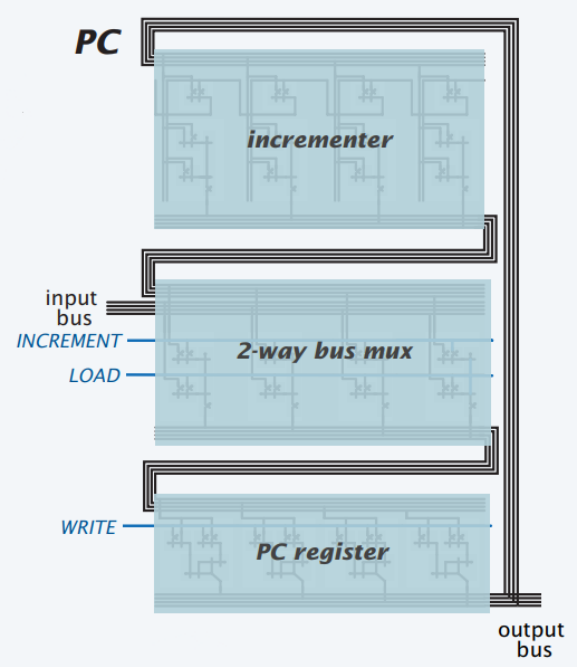
b. PC interface is like



Where PC holds an address and supports 3 control wires:

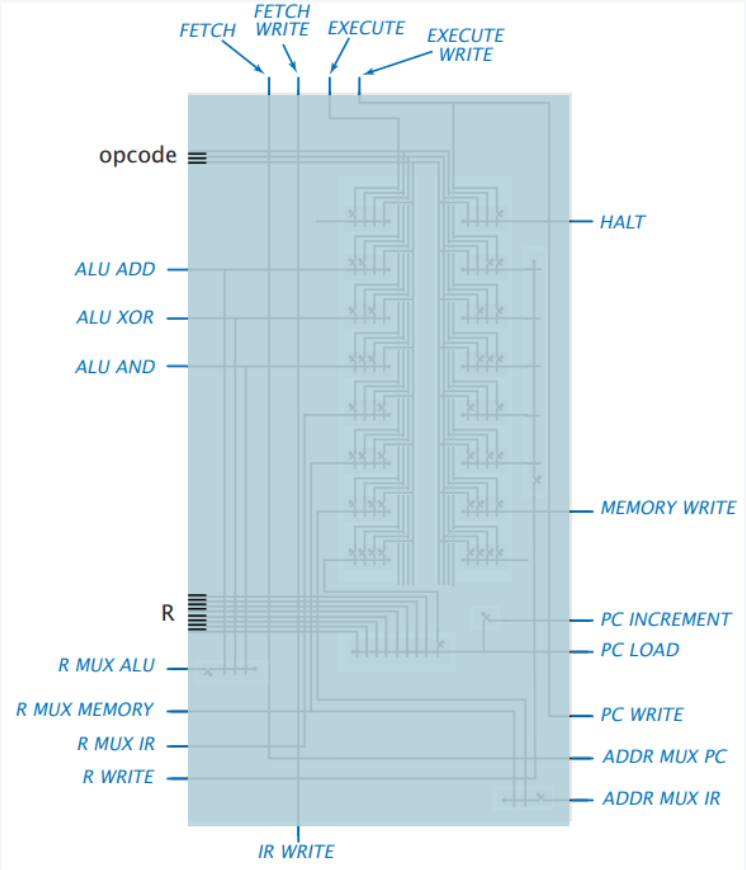
i. INCREMENT – Add 1 to value when WRITE becomes 1
ii. LOAD – Set value from input bus when WRITE becomes 1
iii. WRITE – Enable PC address to change as specified

c. A more detailed interface with connection is shown below

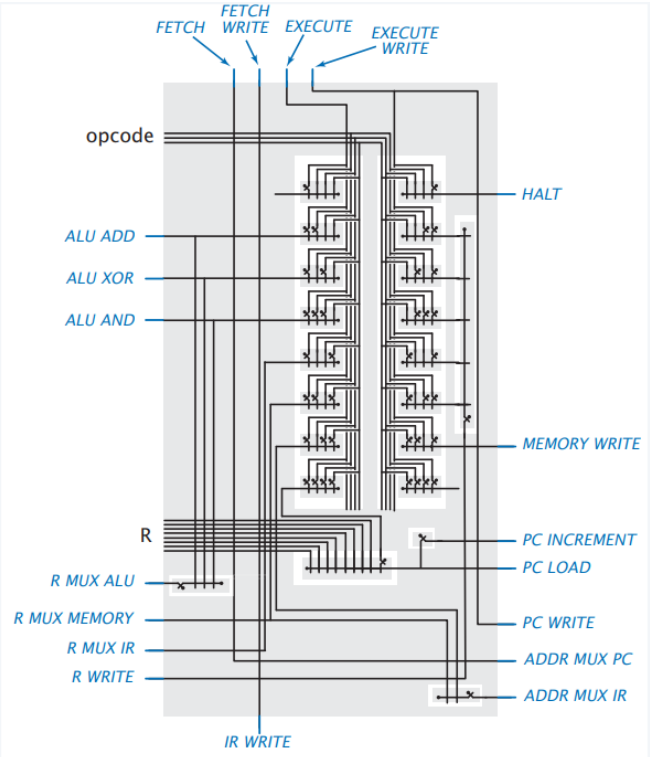


21. Implement Control (one last and simple combinational circuit)
    a. Control would take
        i. control wires from clock and
        ii. data lines from IR (opcode) and
        iii. R (data values) as input.
    b. Control would provide
        i. 15 control wires to different modules in CPU as output

c. Interface shown below



d. Control circuit



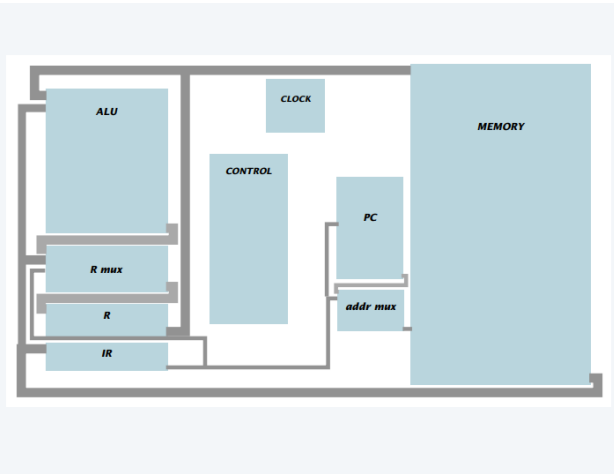
e. Wire sequence explained (study this with the circuit diagram in sight to get clarity)

	FETCH	FETCH WRITE	EXECUTE WRITE
all instructions	ADDR MUX PC	IR WRITE	PC WRITE
	PC INCREMENT*		

* PC LOAD for branch if 0 if R is 0.

instruction	EXECUTE	EXECUTE WRITE
halt	HALT	
add	ALU ADD	R WRITE
	R MUX ALU	
xor	ALU ADD	R WRITE
	R MUX ALU	
and	ALU ADD	R WRITE
	R MUX ALU	
load address	R MUX IR	R WRITE
load	R MUX MEMORY	R WRITE
	ADDR MUX IR	
store	ADDR MUX IR	MEMORY WRITE

22. Connect everything together to make CPU functional
    a. Connecting input/output memory/address buses

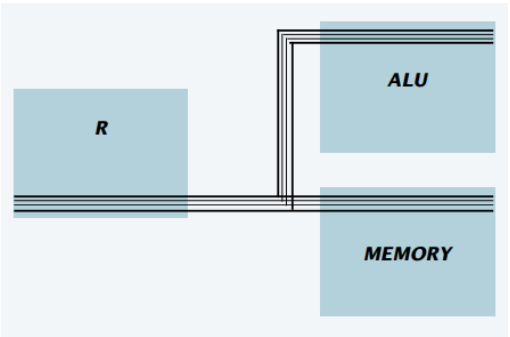


instructions	bus connections
fetch (all)	PC to memory addr
	memory to IR
halt	none
add, and, xor	IR addr to memory addr
	memory to ALU 1
	R to ALU 0
	ALU to R
load address	IR addr to R
load	IR addr to memory addr
	memory to R
store	IR addr to memory addr
	R to memory
branch if zero	IR addr to PC

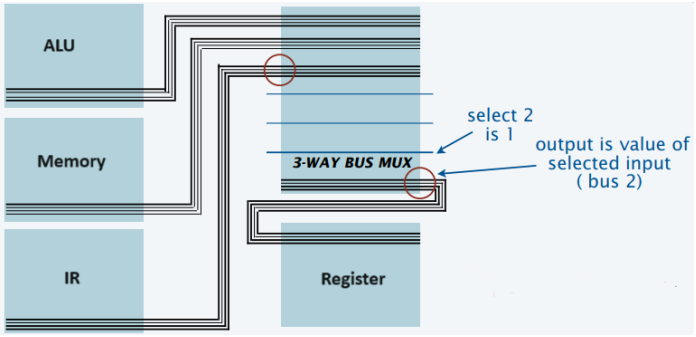
need 2-way bus mux

need 3-way bus mux

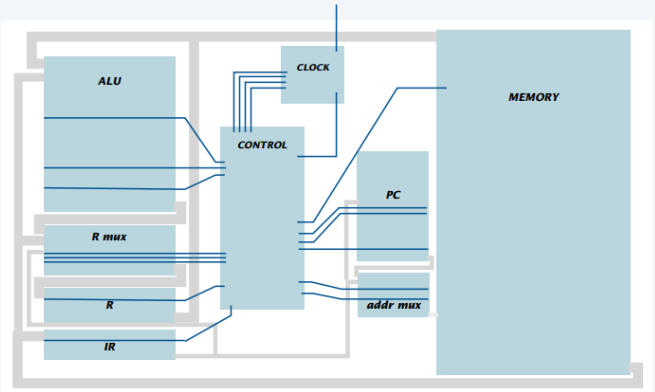
    i. For example register connects to ALU and memory above as



ii. But when ALU, memory and IR connect to register we need a selector switch (bus mux)



b. Connecting control wires (blue ones) as shown below



component	control wires
CLOCK	RUN
	HALT
CONTROL	FETCH
	FETCH WRITE
	EXECUTE
	EXECUTE WRITE
ALU	ADD
	XOR
	AND
R mux	R MUX ALU
	R MUX MEM
	R MUX IR
R	R WRITE
IR	IR WRITE
memory	MEMORY WRITE
PC	PC INCREMENT
	PC LOAD
	PC WRITE
addr mux	ADDR MUX PC
	ADDR MUX IR

c. Final Interface with labels